

Ascend C 算子调试调优

目 录

1 引言：从“能跑”到“跑得快”	3
2 孪生调试：一份源码，两个域	3
2.1 CPU 域调试：gdb 与 printf	3
2.1.1 用 gdb 调试	3
2.1.2 用 printf 调试	4
2.2 NPU 域调试：DumpTensor 与 PRINTF	4
2.3 msdebug：NPU 上的 gdb	4
2.3.1 断点设置	5
2.3.2 单步运行	5
2.3.3 变量打印	5
3 性能调优工具：msprof	5
3.1 上板与仿真两种模式	5
3.2 采集的性能数据类别	5
3.2.1 计算类指令耗时与占比	6
3.2.2 搬运类指令耗时与占比	6
3.2.3 搬运带宽速率	6
3.2.4 资源冲突	6
3.2.5 Cache 命中率	6
3.3 算子调优基本流程	7
4 异常检测工具：mssanitizer	7
4.1 内存异常类型	7
4.1.1 非法读写	7
4.1.2 多核踩踏	7
4.1.3 非对齐访问	8
4.1.4 内存泄漏与非法释放	8
4.2 竞争检测	8
4.2.1 竞争风险的定义	9
5 算子性能优化方法	9
5.1 Tiling 切分优化	9
5.2 核内异步流水线：Double Buffer 机制	10
5.3 Scalar 优化	10
5.3.1 避免阻塞标量计算单元	11
5.4 使用底层 API	11
5.5 ICache 优化	11
5.6 层次化访存优化	11

5.6.1 合理使用 TBuf	11
5.6.2 合并相同生命周期变量	11
5.7 计算资源利用优化	12
5.7.1 大型矩阵 reduce_sum 实现	12
5.7.2 矩阵计算对各 Buffer 的充分利用	12
6 实战案例：定位一个搬运瓶颈	13
6.1 第一步：用 msprof 采集性能数据	13
6.2 第二步：分析瓶颈	13
6.3 第三步：选择优化手段	13
6.4 第四步：重新采集并对比	13
7 本章你将学会	14
8 要点速查	14
9 小结	15

1 引言：从“能跑”到“跑得快”

算子开发完成后，你会面临三个层次的问题：算子能不能跑通？跑出来的结果对不对？跑得够不够快？这三个问题对应三个递进的目标：**功能可运行、精度正确、性能极致**。

在传统的 CPU 编程中，你可以用 `printf` 打印中间值，用 `gdb` 下断点单步执行，排错门槛很低。但在昇腾 AI 处理器上，算子运行在 AI Core 的独立执行单元中，Host 侧无法直接窥探核函数内部状态，传统的调试手段大多失效。因此，CANN 提供了一整套专门的调试调优工具链。

直觉 不妨把算子调试想象成“给黑箱装仪表”。CPU 上的程序是透明的玻璃箱，你随时可以看里面发生了什么；NPU 上的算子是不透明的金属箱，你只能通过专门设计的探针（调试工具）来读取内部状态。孪生调试的核心思想是：先用 CPU 这个透明箱把逻辑调对，再放到 NPU 金属箱里调性能，这样大部分逻辑错误可以在低成本环境下发现。

本章围绕四块内容展开：算子调试工具、算子调优工具、算子异常检测工具、昇腾算子性能优化方法。我们会沿着“发现问题,定位问题,解决问题”的顺序逐一展开。

2 孪生调试：一份源码，两个域

孪生调试（Twin Debugging）是 Ascend C 提供的一种调试方法论：相同的算子源码可以在 CPU 域调试精度，在 NPU 域调试性能。两个域共用同一份代码，但运行环境和可用的调试手段不同。

维度	CPU 域	NPU 域
运行位置	Host CPU	昇腾 AI Core
调试目标	精度正确性	性能与上板行为
调试手段	<code>gdb</code> , <code>printf</code>	<code>DumpTensor</code> , <code>PRINTF</code> , <code>msdebug</code>
成本	低，无须真实芯片	高，需要真实芯片或仿真器

直觉 孪生调试就像“先在沙盘上推演，再上战场实战”。CPU 域是沙盘，便宜、快速、可反复试错，用来验证算法逻辑是否正确；NPU 域是战场，环境真实，用来检验上板后的性能和精度。因为两边用同一份代码，你在沙盘上调对的逻辑可以直接拿到战场上用。

2.1 CPU 域调试：gdb 与 printf

CPU 域调试门槛最低，可以用两种方式观察算子行为。

2.1.1 用 gdb 调试

CPU 域下算子以子进程方式启动，可以用 `gdb` 附加到子进程上调试。有两种场景：

1. **调试单个子进程**：直接用 `gdb` 启动宿主程序，子进程继承父进程的调试状态。
2. **调试多个子进程**：当算子启动多个子进程时，需要使用 `set follow-fork-mode child` 和 `set detach-on-fork off` 让 `gdb` 跟随特定的子进程。

多子进程调试是核函数并行执行的常见场景。Ascend C 在 CPU 域会为每个 block 拉起一个子进程来模拟多核并行，调试时你需要明确关注哪一个 block 的行为。

2.1.2 用 printf 调试

在代码中直接编写 printf(...) 来观察数值的输出，是最直接的调试方式。

关键注意：NPU 模式下目前不支持 printf 语句，所以需要添加内置宏 __CCE_KT_TEST__ 予以区分，让同一份代码在两个域都能编译通过：

```
#ifdef __CCE_KT_TEST__
    printf("debug value: %f\n", val);
#endif
```

逐行解析：

1. #ifdef __CCE_KT_TEST__：当宏 __CCE_KT_TEST__ 被定义时（即 CPU 域编译），编译器才会编译下面的语句。
2. printf("debug value: %f\n", val);：在 CPU 域运行时打印变量 val 的值。
3. #endif：条件编译结束。在 NPU 域编译时，这段代码会被完全忽略。

直觉 把 __CCE_KT_TEST__ 理解成一道“域门”。CPU 域编译时门是开的，printf 能走进来发挥作用；NPU 域编译时门是关的，printf 被挡在外面不参与编译。这样一份代码就能在两个域都正常通过编译。

2.2 NPU 域调试：DumpTensor 与 PRINTF

NPU 域无法直接使用标准 printf，需要通过专门的接口输出调试信息。核心步骤如下：

1. **设置环境变量：** 设置打开 DUMP 开关的环境变量。
2. **修改 CMakeLists.txt：** 在算子工程 op_kernel 目录下的 CMakeLists.txt 文件，首行增加编译选项，打开 DUMP 开关。
3. **调用接口：** 在算子 Kernel 侧实现代码中需要输出日志信息的地方调用 DumpTensor 接口或者 PRINTF 接口打印相关内容。

DumpTensor 用于输出张量数据的完整内容，适合检查中间计算结果的精度。PRINTF 则用于输出简单的日志信息，类似 C 语言的 printf 但在 NPU 域可用。

DUMP 开关是一个全局编译选项，打开后编译器会在算子二进制中插入数据导出代码。调试完成后记得关闭开关，否则会影响上板性能。

2.3 msdebug: NPU 上的 gdb

在上板调试时，可以使用 CANN 提供的算子调试工具 msdebug。msdebug 是面向算子开发场景的代码调试工具，支持在昇腾系列 AI 处理器上，对 Ascend C 开发的算子核函数，进行如断点设置、单步运行、变量打印等功能调试，协助算子开发者高效完成代码编写与功能纠正。

直觉 msdebug 就是“NPU 上的 gdb”。它把用户输入的调试命令翻译成机器语言，并完成程序控制。在 CPU 上你用 ptrace API 控制进程；在算子核函数调试中，msdebug 通过驱动层的调试通道，调用任务管理器提供的 Debug API 实施对 AI 处理器的控制，实现对算子核函数的程序控制功能。

msdebug 的核心功能有三种：

2.3.1 断点设置

msdebug 支持三种设置断点的方式：

1. **按函数名：** 根据函数名在函数入口处设置断点。
2. **按代码行：** 在代码源文件的特定代码行设置断点。
3. **按地址：** 对特定内存地址设置断点。

2.3.2 单步运行

在算子程序进入暂停状态后，msdebug 支持让算子核函数跳过当前代码行，停在逻辑上应执行的后一行代码前。这对应 gdb 中的 next 命令。

2.3.3 变量打印

在算子程序处于暂停状态时，msdebug 支持打印断点代码行作用域内的变量值。这对应 gdb 中的 print 命令。

3 性能调优工具：msprof

msprof 是 CANN 提供的算子调优工具，可用于采集和分析昇腾 AI 处理器算子运行阶段的关键性能指标，如计算类指令耗时、搬运类指令耗时、搬运带宽、资源冲突、Cache 命中率等信息。

msprof 的输出不仅包含数值指标，还支持根据昇腾 CANN 软件栈中提供的仿真能力，给出算子在仿真状态下的指令流水线图、算子代码热点图、指令到代码行映射表等性能数据。开发者可根据调优工具输出的性能数据，快速定位软、硬件性能瓶颈，提升算子性能分析的效率。

3.1 上板与仿真两种模式

msprof 支持两种运行模式进行调优：

- **上板 (On-Board)：** 算子运行在真实的昇腾 AI 芯片上，性能数据反映真实硬件行为。
- **仿真 (Simulation)：** 算子运行在 CANN 提供的昇腾性能仿真器上，无须真实的昇腾 AI 处理器即可依靠 CPU 模拟出昇腾 AI 芯片的计算过程。

直觉 上板是“实弹演习”，数据真实但需要硬件；仿真是“沙盘推演”，不需要芯片就能预估性能。开发阶段可以先用仿真快速迭代，再上板做最终验证。

3.2 采集的性能数据类别

msprof 采集的性能数据可以分为以下几类，理解每一类的含义是性能优化的前提。

3.2.1 计算类指令耗时与占比

计算类指令（Compute Instruction）指涉及昇腾芯片架构中矩阵和向量计算单元的相关指令。计算类指令是实现算子逻辑的关键，这类耗时属于算子**不可掩盖的耗时**，也就是必须花的时间。

不可掩盖的含义是：无论怎么优化数据搬运，计算本身的时间都无法消除。因此优化的重点不是减少计算耗时，而是让搬运与计算并行，把可掩盖的搬运时间藏起来。

3.2.2 搬运类指令耗时与占比

搬运类指令（Move Instruction）指算子所需的相关数据在昇腾芯片上的不同内存间搬运的过程。一个算子理想情况下是计算单元一直在持续地进行计算，因此搬运类指令的耗时属于**可被掩盖耗时**。

通过将搬运数据的过程和计算的过程并行起来，可以降低计算单元等待数据准备好的时间，提高算子的整体性能。这是性能优化最大的空间所在。

3.2.3 搬运带宽速率

算子的数据搬运效率可以通过**搬运带宽速率**（Move Bandwidth）直观呈现。搬运带宽速率表示在单位时间内搬运的数据量，速率越高，在相同数据量下所耗费的时间更少。

3.2.4 资源冲突

对于昇腾设备上的部分内存单元（具体单元取决于昇腾芯片型号）中存在 **bank** 和 **bank group** 的内存架构。一个 bank 由多个最小内存访问单元 block 组成，多个 bank 则组成一个 bank group。冲突会发生在：

- 同一个 **bank group** 会有**读读冲突**。
- 同一个 **bank** 中会有**读写冲突**。

直觉 可以把 bank 想象成银行的柜台，bank group 是一组柜台。两个请求同时读同一组柜台的不同柜台，会产生组间干扰（读读冲突）；两个请求同时对同一柜台一读一写，会产生直接冲突（读写冲突）。冲突会让访存请求排队等待，拉低有效带宽。

3.2.5 Cache 命中率

昇腾芯片架构中基于极致性能的考虑设计了 **Cache** 缓存以提高数据搬运速率。在 GM/HBM 上包含了 L2 缓冲区的缓存，可以极大提升数据从 GM/HBM 到 L1 或者 UB 的速率。

提高算子计算时对于同一块在 GM/HBM 上的数据复用率，可以增加 Cache 命中率，进而缩短数据搬运所耗费的时间。

Cache 命中率优化的核心是“数据局部性”。如果一个数据块被多次使用，应该让它在 Cache 里尽量待得久一点。Tiling 切分时让一个 tile 内的计算尽量复用同一块数据，就是基于这个原理。

3.3 算子调优基本流程

算子调优的基本流程可以概括为：

1. 用 msprof 采集算子上板或仿真的性能数据。
2. 分析指令流水线图和代码热点图，定位性能瓶颈所在（计算瓶颈还是搬运瓶颈）。
3. 针对瓶颈选择优化手段（Tiling 切分、Double Buffer、层次化访存等）。
4. 重新用 msprof 采集数据，对比优化前后的指标。
5. 重复迭代直到性能达标。

4 异常检测工具：mssanitizer

mssanitizer 是基于昇腾芯片架构的一个异常检测工具，包含了**内存检测**和**竞争检测**两个功能。

内存检测工具可以在用户开发算子过程中，协助定位并解决诸如内存非法读写、多核踩踏、非对齐访问、内存泄漏、非法释放等内存问题。同时内存检测工具也支持对 CANN 软件栈的内存检测，帮助用户定界软件栈内存异常发生的模块。竞争检测工具可以协助用户定位由于竞争风险可能导致的数据竞争问题。

直觉 在昇腾芯片架构上，为了极致性能考虑设计了多级内存。在 Kernel 执行过程中，数据会在不同内存之间进行搬运，开发者需要小心地处理搬运的范围与时机。当我们聚焦某一块特定内存时，内存检测的基本原理就是当内存事件发生时，通过特定算法检查相关内存的状态，从而确定内存事件是否合法。

4.1 内存异常类型

4.1.1 非法读写

非法读写 (Illegal Access) 异常信息的产生是由于算子程序中，通过读或写的方式访问了一块未分配的 (unaccessible) 内存。此错误一般发生在 GM/HBM 上，由于用户 GM 分配的大小与实际算子程序中访问的范围不一致导致。

例 假设内存中 0x50 到 0x7f 范围的内存处于未分配状态，此时针对 0x00 地址发生长度为 0xa0 的读或写事件。

访问范围是 0x00 到 0x9f (起始地址 0x00 加上长度 0xa0)，与未分配区域 0x50 到 0x7f 有重叠。

重叠部分长度 = $0x7f - 0x50 + 1 = 0x30$ (十进制 48 字节)。

因此会在未分配的内存位置产生一个长度为 0x30 的非法读写异常。修复方法是扩大 GM 分配大小，使其覆盖实际访问范围。

4.1.2 多核踩踏

昇腾芯片中有多个 AI Core 芯片同时参与运算，这些 AI Core 芯片会在计算过程中从 GM 上搬入或搬出数据。当没有显式地进行核间同步时，如果各个核之间访问的 GM 内存存在重叠则会发生**多核踩踏** (Multi-Core Overlap) 问题。

我们通过**所有者（Owner）**的概念来保证多核之间不会发生踩踏问题：当一块内存被某一个核写入过后，这块内存就由该核所有。当其他核对这块内存进行读写访问时就会产生 out of bounds 异常。

当两个 AI Core 访问的内存区域发生重叠时，重叠的内存区域就可能会发生多核踩踏异常。

多核踩踏的根源是“没有同步就共享写”。解决办法是要么用同步原语显式做核间同步，要么保证各核访问的 GM 区域互不重叠（通过 Tiling 切分让每个 block 独占一段 GM）。

4.1.3 非对齐访问

昇腾芯片上包含多种类型的内存，当通过 **DMA** 接口进行访问时，不同类型的内存存在不同芯片上有不同的最小访问粒度。当访问的内存地址或长度与最小访问粒度不对齐时，会发生数据异常或 AI Core 异常等问题。

访问对齐检测可以在对齐问题发生时输出对齐异常信息。

4.1.4 内存泄漏与非法释放

内存泄漏（Memory Leak）：内存检测工具可以检测出 Device 侧的内存泄漏问题，这些问题通常是开发者没有正确释放使用 ACL 接口申请的内存导致的。由于片上内存目前不存在内存分配的概念，因此内存泄漏只能出现在 GM/HBM 上。通过指定命令行参数 `--leak-check=on` 可以开启内存泄漏检测。

非法释放（Illegal Free）：非法释放是指对一个未分配的地址或者已释放的地址进行了释放操作，一般发生在 GM 上。与 CPU 侧类似，GM 上的地址在申请与释放时也是整块进行操作，释放时需要指定申请时 `aclrtmalloc` 接口返回的地址。

非法释放有两种典型错误：

- **illegal free**： 对一个不指向内存块首地址的指针调用释放接口会产生此错误。
- **double free**： 对一个已释放的地址重复调用释放接口会产生此错误。

直觉 GM 内存的申请和释放就像停车场的车位管理。`aclrtmalloc` 给你一个车位号（首地址），离开时必须用这个车位号退车位（释放）。如果你用一个不存在的车位号退，停车场会报 illegal free；如果你退了同一个车位两次，停车场会报 double free。

4.2 竞争检测

竞争检测工具是一个用于检测运行时共享内存同时访问风险的工具，此工具主要用途是帮助用户识别到内存数据的竞争访问风险。

在昇腾芯片架构下，AI Core 的片上内存可以被多个流水线（PIPE）访问，片上内存经常被用作临时缓冲区保存正在处理的数据。由于这些数据可以同时被多个流水线访问，算子程序没有正确处理流水线同步时就可能会导致数据竞争问题。

4.2.1 竞争风险的定义

竞争风险（Race Condition）是指两个事件（其中至少有一个为写事件）尝试访问同一块内存时，基于两个事件的相对顺序发生的非预期访问结果。有三种典型类型：

1. **WAW（Write-After-Write）**：当两个内存事件尝试向同一块内存写入时，就会存在这种风险，而内存访问结果取决于两个内存事件的相对顺序。先写的值会被后写的值覆盖，如果实际顺序与预期相反，结果就错了。
2. **WAR（Write-After-Read）**：当两个内存事件尝试访问同一块内存，一个执行读操作，另一个执行写操作时，就会存在这种风险。在这种情况下，写操作事件会在读操作事件提交之前修改内存的值，读到的可能是脏数据。
3. **RAW（Read-After-Write）**：当两个内存事件尝试访问同一块内存，一个执行写操作，另一个执行读操作时，就会存在这种风险。在这种情况下，写操作事件在读操作事件之前执行，所以返回给读操作事件的内存值并不是其原始值。

直觉 竞争风险的共同点是“两个事件争同一块内存，至少一个是写”。WAW 是“两次写打架”，最后值不确定；WAR 是“还没读完就写了”，读到的可能是新值；RAW 是“先写了再读”，读到的是新值。三种风险的解决办法都是插入同步屏障，强制事件按预期顺序执行。

5 算子性能优化方法

对于昇腾算子的性能优化，可以从四个角度进行和展开：

优化角度	主要内容
算子计算流程优化	Tiling 切分优化、核内异步流水线（Double Buffer）
代码实现优化	Scalar 优化、避免阻塞标量计算单元、使用底层 API、ICache 优化
层次化访存优化	合理使用 TBuf、合并相同生命周期的数据
计算资源利用优化	大型矩阵 reduce_sum 实现、矩阵计算对各级 Buffer 的充分利用

直觉 这四个角度可以理解成“由粗到细”的优化层次。计算流程优化是宏观的“怎么切数据、怎么排流水线”；代码实现优化是微观的“怎么写代码”；层次化访存优化是“怎么管 Buffer”；计算资源利用优化是“怎么榨干硬件算力”。实际优化时应该从宏观到微观逐层展开。

5.1 Tiling 切分优化

Tiling（切分）通常是指对计算进行并行计算切分，以便可以启动并行实例进行并行计算。Tiling 切分有如下几个基本原则：

1. **负载均衡**：Tiling 需要考虑部署环境，也就是并行计算单元的个数，切分后的计算尽量负载均衡，避免有忙有闲，浪费资源。
2. **用满资源**：Tiling 尽量用满所有可用资源，不要有闲置资源。
3. **考虑双缓冲**：核内 Tiling 的块要考虑双缓冲机制（在 Ascend C 中分配 2 块流水线用的 TQue 缓冲区）。

4. **块尽量大**：核内 Tiling 的块要尽量大，一方面可以降低标量计算单元的压力（循环次数少），另一方面有利于 SIMD 发挥算力，降低指令启动开销。

直觉 把 Tiling 想成“分蛋糕给小朋友”。负载均衡是“每个小朋友分到的蛋糕一样多”，避免有人撑死有人饿着；用满资源是“所有小朋友都参与，不要有人闲着”；考虑双缓冲是“给每个小朋友两个盘子，一个在吃，另一个在装下一块”；块尽量大是“每块蛋糕切大一点，减少切蛋糕的次数”。

例 假设有一个长度为 4096 的向量要做加法，部署环境有 8 个并行计算单元。

如果 Tiling 切分为 8 块，每块 512 个元素：8 个计算单元各处理一块，负载完全均衡。

如果 Tiling 切分为 10 块，每块约 410 个元素：前 8 个块分给 8 个计算单元后还剩 2 块，需要等某计算单元空闲才能处理，造成有忙有闲，负载不均衡。

核内 Tiling 再把每块 512 个元素切成多个 tile。如果切 8 个 tile，每个 tile 64 个元素，加上双缓冲（BUFFER_NUM=2），实际每个 TQue 缓冲区持有 32 个元素。块切得越大，外层循环次数越少，标量计算单元的压力越低。

5.2 核内异步流水线：Double Buffer 机制

核内异步流水线其实就是指前文中提到的双缓冲（Double Buffer）机制。Ascend C 提供了编程范式会帮助自动实现流水线并行能力。

直觉 没有 Double Buffer 时，数据搬运和计算是串行的：搬一组数据,算一组数据,搬下一组,算下一组。计算单元在搬运时空闲，搬运单元在计算时空闲。

有了 Double Buffer 后，分配两块 TQue 缓冲区，当计算单元在算缓冲区 A 的数据时，搬运单元可以把下一组数据搬进缓冲区 B。等 A 算完了，B 也搬好了，直接切换到 B 计算，同时往 A 搬下下组数据。这样搬运和计算就能并行起来，计算单元几乎不用等待。

Ascend C 的编程范式（CopyIn, Compute, CopyOut 三段式）会自动帮你管理这两块缓冲区的切换，开发者只需要在 pipe.InitBuffer 时指定 BUFFER_NUM = 2 即可。

Double Buffer 的本质是“用空间换时间”。多分配一块缓冲区占用了更多 UB 内存，但换来的是计算单元的持续工作。当 UB 内存紧张时，需要在缓冲区数量和可用空间之间做权衡。

5.3 Scalar 优化

标量计算单元（Scalar Unit）的任务是完成地址计算、循环控制、条件控制、指令发射，期望的模式是无阻塞的，直到完成所有指令发射。一旦标量计算单元被阻塞，整个算子的指令发射都会停滞。

标量计算单元常用的优化技术与对 CPU 的取指优化技术类似，主要有：

- **循环内重复计算消除**：把循环中每次都重复计算且结果相同的表达式提到循环外。

- **计算强度削减：** 用加法替代乘法等强度更低的运算。
- **计算提取至板外进行：** 把不依赖运行时数据的计算放到 Host 侧预先算好。

5.3.1 避免阻塞标量计算单元

避免标量计算单元参与数据计算，可以避免阻塞标量计算单元。标量计算单元的职责是发射指令，不是做数据计算。如果在算子代码中让标量单元去做大量的数据运算，它会顾不上发射指令，导致向量计算单元和矩阵计算单元“饿着肚子等指令”。

直觉 标量计算单元就像厨房里的“传菜员”，它的工作是把订单（指令）传给后厨的厨师（向量/矩阵计算单元）。如果你让传菜员也去做菜（数据计算），他就没时间传菜了，后厨的厨师就只能干等着。正确做法是传菜员只管传菜，做菜交给厨师。

5.4 使用底层 API

考虑使用更底层、更完整发挥硬件优势的 API 接口进行计算。Ascend C 提供了多层 API，高层 API 封装好但可能隐藏了硬件细节，底层 API 更贴近硬件但使用更复杂。在性能敏感的场景下，应该考虑使用底层 API 来榨取硬件性能。

5.5 ICache 优化

ICache（指令缓存）用于缓存即将执行的指令。对上板运行的数据结果统计分析可发现，会存在 **ICache miss**（缓存未命中）的情况。ICache miss 会导致指令需要重新从低速内存加载，拉低执行效率。

ICache miss 的优化建议措施有如下两点：

1. **减小代码段大小：** 让代码段大小至 ICache 能放下，避免反复换入换出。
2. **热点分支前置：** 执行次数多的分支代码放于分支最前，让 ICache 更容易命中热点代码。

ICache 优化的思路和 CPU 上的指令缓存优化一致。核心原则是“常用的东西放在手边”，减少缓存未命中带来的访存延迟。

5.6 层次化访存优化

层次化访存优化主要涉及对流水线缓冲区的分配以及访存，包括合理使用 TBuf 和合并相同生命周期的数据两方面。

5.6.1 合理使用 TBuf

在使用 **TBuf** 数据结构时会申请 **TMP Buffer**（临时变量 Buffer）。对该临时变量反复使用时不需要反复申请并释放，而是将这个临时变量保存到算子类成员变量中，以便随时调用。

这样做的好处是：避免每次使用临时变量时都走一遍申请释放流程，减少 Scalar 操作开销和内存碎片。

5.6.2 合并相同生命周期变量

对拥有相同生命周期的数据，可以对其分配的 **TQue** 进行合并，降低 Scalar 操作开销。

例 以 $Z = X + Y$ 这样的计算为例，X 和 Y 操作数生命周期实际上是相同的：都需要各自搬运一个操作数到 UB，然后开始计算，计算完成同时释放。

如果不合并，需要申请两个 TQue，每个 TQue 一个 Buffer：

```
pipe.InitBuffer(inQueueX, BUFFER_NUM, tileLength * sizeof(half));
pipe.InitBuffer(inQueueY, BUFFER_NUM, tileLength * sizeof(half));
auto xLocal = inQueueX.AllocTensor<half>();
auto yLocal = inQueueY.AllocTensor<half>();
```

合并后，只申请一个 TQue，一个 Buffer 存放 xLocal 和 yLocal，通过 xLocal + offset 获得 yLocal 并使用：

```
pipe.InitBuffer(inQueueXY, BUFFER_NUM, 2 * tileLength * sizeof(half));
auto xLocal = inQueueXY.AllocTensor<half>();
auto yLocal = xLocal[tileLength];
```

逐行解析：

1. `pipe.InitBuffer(inQueueXY, BUFFER_NUM, 2 * tileLength * sizeof(half))`：申请一个 Buffer，大小是原来的两倍，能同时容纳 X 和 Y。
2. `auto xLocal = inQueueXY.AllocTensor<half>()`：从队列分配张量，xLocal 指向 Buffer 起始位置。
3. `auto yLocal = xLocal[tileLength]`：通过偏移量 tileLength 获得 yLocal，yLocal 指向 Buffer 的后半段。

合并后少了一次 InitBuffer 调用和一次 AllocTensor 调用，Scalar 操作开销减半。代价是 Buffer 占用空间不变（两段连续内存），但管理更简洁。

5.7 计算资源利用优化

计算资源利用优化着重关注数据搬运以及数据复用，对矩阵运算会有比较大的提升。主要包含两个方面。

5.7.1 大型矩阵 reduce_sum 实现

大型矩阵的 **reduce_sum**（归约求和）如果用标量逐元素累加，性能会非常差。主要思路是通过构造辅助矩阵，利用 **matmul**（矩阵乘法）计算单元来实现 reduce_sum，把标量计算转化为矩阵计算，充分发挥矩阵计算单元的高吞吐优势。

5.7.2 矩阵计算对各 Buffer 的充分利用

在进行矩阵计算时要想做到对各层级缓冲区充分利用，需要考虑多核复用内存，并充分利用 L2、L1、L0 的数据复用能力。

两种全载策略：

1. **L0 全载计算**：当一个矩阵小到足以放到 L0A/L0B 的 64KB 空间时，则直接进行全量载入。这时另外一个矩阵整体只需要进入 L0 一次就完成全部计算，避免了反复搬运。
2. **L1 全载计算**：当一个矩阵小到足以放到 L1 的 504KB 空间时，则可以直接全载 L1。这样另外一个矩阵（高性能格式场景）只需要进入一次 L0 则完成计算。

直觉 全载策略的核心是“一次搬够，反复使用”。以矩阵乘法 $C = A \times B$ 为例，如果 A 是小矩阵能全载到 L0，那么 B 的每一块进来后都能和完整的 A 相乘，A 不需要反复从高级内存搬运。矩阵越小，能全载的层级越深（L0 比 L1 更接近计算单元），复用收益越大。

6 实战案例：定位一个搬运瓶颈

假设你开发了一个向量加法算子，上板后发现性能不达预期。下面是用 msprof 定位瓶颈并优化的完整过程。

6.1 第一步：用 msprof 采集性能数据

在算子运行时用 msprof 采集性能数据，重点关注以下指标：

```
msprof --application=./run_op --output=./prof_data
```

采集到的数据示例：

指标	优化前	目标
计算类指令耗时	30 us	30 us（不可掩盖）
搬运类指令耗时	80 us	30 us（藏到计算后）
搬运带宽速率	4 GB/s	12 GB/s
资源冲突次数	15 次	0 次
Cache 命中率	40%	80%

6.2 第二步：分析瓶颈

从数据看，搬运类指令耗时（80 us）远大于计算类指令耗时（30 us），说明算子是“搬运受限”的。计算单元大部分时间在等数据，资源冲突 15 次和 Cache 命中率 40% 也印证了访存效率低下。

6.3 第三步：选择优化手段

针对搬运瓶颈，可以采取以下措施：

1. **开启 Double Buffer**：在 pipe.InitBuffer 中使用 BUFFER_NUM = 2，让搬运与计算并行。
2. **增大 Tiling 块**：减少 tile 数量，增大每块大小，降低标量计算单元循环开销。
3. **合并相同生命周期变量**：把 X 和 Y 的 TQue 合并为一个，减少搬运次数。
4. **检查对齐**：确保 DMA 访问地址和长度与最小访问粒度对齐，消除资源冲突。

6.4 第四步：重新采集并对比

优化后重新用 msprof 采集数据，搬运类指令耗时从 80 us 降到 30 us（与计算并行后被掩盖），Cache 命中率从 40% 提升到 80%，资源冲突降为 0 次。算子整体性能提升约 2 倍。

这个案例的关键在于“用数据驱动优化”。不要凭感觉优化，而要用 *msprof* 采集的客观数据来定位瓶颈，选择对应的优化手段，再用 *msprof* 验证效果。这是一个迭代过程，可能需要多轮才能达到目标性能。

7 本章你将学会

1. 理解孪生调试方法，区分 CPU 域和 NPU 域的调试手段。
2. 使用 gdb 和 printf 在 CPU 域调试算子精度，用 `__CCE_KT_TEST__` 宏区分编译模式。
3. 使用 DumpTensor、PRINTF 和 msdebug 在 NPU 域调试算子上板行为。
4. 使用 msprof 采集性能数据，分析计算类、搬运类、冲突、Cache 命中率等指标。
5. 使用 mssanitizer 检测内存异常和数据竞争，理解五种内存异常和三种竞争风险。
6. 应用 Tiling 切分、Double Buffer、Scalar 优化、ICache 优化、层次化访存等性能优化手段。

8 要点速查

工具/概念	用途	关键点
孪生调试	CPU 域调精度，NPU 域调性能	同一份源码在两个域运行
gdb	CPU 域子进程调试	follow-fork-mode child 跟随子进程
printf + <code>__CCE_KT_TEST__</code>	CPU 域打印	NPU 域不支持 printf，用宏区分
DumpTensor	NPU 域输出张量	需设置 DUMP 环境变量和 CMakeLists 选项
PRINTF	NPU 域日志输出	简单日志，类似 printf
msdebug	NPU 域代码调试器	断点/单步/变量打印，类似 gdb
msprof	性能调优	采集耗时/带宽/冲突/Cache，支持上板/仿真
mssanitizer	异常检测	内存检测 + 竞争检测
非法读写	访问未分配内存	GM 分配大小与访问范围不一致
多核踩踏	核间未同步访问重叠 GM	所有者机制，out of bounds 异常
非对齐访问	地址/长度与最小粒度不对齐	DMA 接口访问时发生
内存泄漏	Device 侧未释放 ACL 内存	--leak-check=on 开启检测
非法释放	非首地址释放或重复释放	illegal free / double free
WAW/WAR/RAW	三种竞争风险	插入同步屏障解决
Tiling 四原则	切分优化	均衡/用满/双缓冲/块大
Double Buffer	搬运与计算并行	BUFFER_NUM = 2，编程范式自动实现

工具/概念	用途	关键点
Scalar 优化	避免阻塞标量单元	只做地址/循环/条件/指令发射
ICache 优化	减少缓存未命中	减小代码段/热点分支前置
TBuf 复用	临时变量保存为成员	避免反复申请释放
合并 TQue	相同生命周期变量合并	xLocal + offset 获得 yLocal
L0/L1 全载	小矩阵全量载入	L0: 64KB, L1: 504KB

9 小结

调试调优是算子开发从“能跑”到“跑得快”的关键环节。孪生调试方法让你在 CPU 域调精度、NPU 域调性能，一份源码两域共用；msdebug 帮你在 NPU 上做断点调试；msprof 帮你定位性能瓶颈，区分计算耗时与搬运耗时；mssanitizer 帮你发现内存异常和数据竞争。

性能优化手段从 Tiling 切分、Double Buffer、Scalar 优化到层次化访存、L0/L1 全载计算，层层递进。核心原则是：**先正确，再快速**。先用调试工具保证功能正确，再用调优工具定位瓶颈，最后用优化手段提升性能。整个过程中，msprof 采集的客观数据是优化决策的唯一依据。

下一章将进入大模型算子优化的实战，把本章的调试调优方法应用到具体场景中。

讲义基于华为 CANN「第五章 Ascend C 算子调试调优」课程内容编写